

Satellitenavigation

Institutsprojekt

Code-Dokumentation

Ueberarbeitete, dokumentierte und um Aufgabenblock 5 (Genauigkeit)
erweiterte Fassung des Projekts.

RWTH Aachen · Lehrstuhl fuer Navigation · Sommersemester 2026
Stand: 06.07.2026

1. Ueberblick

Dieses Projekt wertet zwei GNSS-Rohdatenaufzeichnungen (RINEX Obs, statisch und dynamisch) sowie eine Navigationsdatei (RINEX Nav) aus und bearbeitet damit die vier Aufgabenblöcke des Institutsprojekts:

Block	Thema	Code-Ordner
2	Observablen	Observation/
3	Satellitenorbits	Orbit/
4	Positionsbestimmung	Positionsbestimmung/
5	Genauigkeit	Genauigkeit/ (neu)

Im Rahmen dieser Uebearbeitung wurden:

- mehrere Laufzeitfehler in Aufgabenblock 4 behoben (Details in Abschnitt 4),
- Aufgabenblock 5 (Genauigkeit) neu implementiert (Details in Abschnitt 5),
- der Code so restrukturiert, dass jeder Aufgabenblock eine eigene `run(...)`-Funktion besitzt, die zentral aus `main.py` aufgerufen wird,
- alle Funktionen mit Docstrings versehen,
- nicht mehr benötigte Dateien entfernt und `__pycache__` aus der Versionsverwaltung genommen.

2. Projektstruktur & Kontrollfluss

`main.py` lädt die Rohdaten genau einmal und ruft anschliessend die vier Block-Hauptfunktionen der Reihe nach auf. Jeder Aufgabenblock kapselt seine Teilaufgaben (a, b, c, ...) in eigenen Modulen und stellt eine Funktion `run(...)` als einheitlichen Einstiegspunkt bereit:

```
main.py
|
|-- Observation.observation_main.run(data_stat, data_dyn)      # Block 2
|-- Orbit.orbit_main.run(data_nav)                             # Block 3
|-- Positionsbestimmung.position_main.run(data_stat,
|    data_dyn, data_nav)                                     -> ergebnisse_4 # Block 4
|-- Genauigkeit.genauigkeit_main.run(data_stat, data_dyn,
|    data_nav, ergebnisse_4)                                # Block 5
```

Die Ergebnisse aus Block 4 (berechnete ECEF-Positionen) werden an Block 5 uebergeben, da Aufgabenblock 5 explizit einen Vergleich mit Abschnitt 4 verlangt.

Ordnerstruktur (Auszug)

```
sat_nav/
|-- main.py                      Einstiegspunkt, ruft die vier Bloecke auf
|-- Observation/
|   |-- observation_c.py .. _g.py Teilaufgaben c-g
|   `-- observation_main.py      Block-Einstiegspunkt
|-- Orbit/
|   |-- orbit_basis.py           gemeinsame Konstanten & Kepler-Loesung
|   |-- orbit_c.py .. _g.py      Teilaufgaben c-g
|   `-- orbit_main.py           Block-Einstiegspunkt
|-- Positionsbestimmung/
|   |-- nutzerposition_a.py      Newton-Verfahren (Aufgabe a)
|   |-- convert_ecef.py         ECEF <-> WGS84 <-> ENU
|   |-- trajektorie.py          Trajektorien-Berechnung & Kartendarstellung
|   `-- position_main.py        Block-Einstiegspunkt
|-- Genauigkeit/
|   |-- ionofrei.py             Zweifrequenz-Ionosphaerenkorrektur
|   |-- genauigkeit_b.py        Statistikvergleich (Mittelwert/Varianz)
|   |-- genauigkeit_c.py        Kartenvergleich der Trajektorien
|   `-- genauigkeit_main.py     Block-Einstiegspunkt
`-- Rohdateien/                RINEX Obs-/Nav-Rohdaten
```

3. Aufgabenblöcke 2 & 3 (Observablen, Satellitenorbits)

Diese beiden Blöcke waren inhaltlich bereits korrekt implementiert. Sie wurden fuer diese Ueberarbeitung lediglich in je eine `*_main.py` mit einer `run(...)`-Funktion verschoben, damit sie aus dem Hauptprogramm einheitlich aufgerufen werden koennen, und mit vollstaendigen Docstrings versehen.

- **Block 2 (Observablen):** Anzahl Satelliten je System, GPS L1-Signal (Pseudorange, Doppler, SNR), Galileo E1/E5-Vergleich, Pseudorange-vs-Traegerphase-Vergleich sowie ein Dopplervergleich zwischen der statischen und der dynamischen Aufzeichnung.
- **Block 3 (Satellitenorbits):** Kepler-basierte Berechnung der Satellitenposition im ECEF- und ECI-Koordinatensystem sowie der Satellitenuhrenkorrektur ueber 24h, fuer einen exemplarisch ausgewaehlten Navigationssatelliten.

Die Kernfunktionen aus `Orbit/orbit_basis.py` und `Orbit/orbit_d.py` (Loesung der Kepler-Gleichung, ECEF-Position, Uhrenkorrektur) werden in Aufgabenblock 4 direkt wiederverwendet - dort traten dadurch zwei der unten beschriebenen Fehler zutage, da diese Funktionen urspruenglich nur fuer einen einzelnen Satelliten getestet waren.

4. Aufgabenblock 4 (Positionsbestimmung) - Fehleranalyse & Korrektur

Der Aufruf von Aufgabe c)-e) in der urspruenglichen `main.py` war fehlerhaft. Aufgabe c) und d) verwendeten faelschlicherweise die Position aus dem synthetischen Test (Aufgabe a/b) anstatt echte, aus den Beobachtungsdaten berechnete Positionen; Aufgabe e) verwendete die statische statt der dynamischen Aufzeichnung. Zudem war die eigentliche Berechnungsfunktion `visualisiere_trajektorie()` beim Aufruf nicht lauffaehig. Im Einzelnen wurden folgende Fehler gefunden und behoben:

Fehler 1 - Falsches Entpacken des Rueckgabewerts.

`berechne_ecef_und_uhr()` liefert vier Werte `(x, y, z, dt_sat)`, in `visualisiere_trajektorie.py` wurde jedoch nur in zwei Variablen `(sat_pos, sat_dt)` entpackt - ein sofortiger `ValueError` beim Ausfuehren.

Behoben: `x, y, z, sat_dt` werden entpackt und per `np.column_stack((x, y, z))` zu einer Nx3-Positionsmatrix zusammengefuehrt (`Positionsbestimmung/trajektorie.py`).

Fehler 2 - `float()`-Konvertierung bei mehreren Satelliten.

`berechne_ecef_und_uhr()` rief auf allen Ephemeriden-Feldern `float(...)` auf. Das funktioniert nur, solange `eph` genau einen Satelliten enthaelt (Block 3). Fuer die Positionsbestimmung wird die Funktion jedoch mit den Ephemeriden mehrerer gleichzeitig

sichtbarer Satelliten aufgerufen - `float()` auf einem mehrelementigen Array wirft einen `TypeError`.

Behoben: Die `float(...)`-Aufrufe wurden entfernt; die Funktion arbeitet nun sowohl fuer einen einzelnen Satelliten (skalare Werte) als auch vektorisiert fuer mehrere Satelliten gleichzeitig (`Orbit/orbit_d.py`).

Fehler 3 - Skalarvergleich in der Kepler-Iteration.

Die Loesung der Kepler-Gleichung in `orbit_basis.berechne_orbit_ebene()` prueft die Konvergenz mit `if diff < 1e-12`. Ist `diff` (mehrere Satelliten gleichzeitig) ein Array statt eines Skalars, wirft dies `"truth value of an array is ambiguous"`.

Behoben: Konvergenzpruefung ueber `np.all(diff < 1e-12)` (`Orbit/orbit_basis.py`).

Fehler 4 - Ungueltige `sel(..., method="nearest")`-Kombination.

`data_nav.sel(sv=liste, time=zeitpunkt, method="nearest")` versucht, die "nearest"-Methode auch auf den textuellen `sv`-Index anzuwenden, was zu `TypeError: unsupported operand type(s) for -: 'str' and 'str'` fuehrt.

Behoben: Zeit- und Satellitenauswahl werden getrennt: zuerst wird pro Satellit der zeitlich naechstgelegene *gueltige* Navigationsdatensatz bestimmt (siehe `_gueltige_nav_daten()` in `trajektorie.py`), erst danach erfolgt die exakte Selektion der sichtbaren Satelliten.

Fehler 5 - Fehlende Satellitensystem-Einschraenkung.

Die Aufgabenstellung schreibt vor, zunaechst nur GPS-Satelliten zu verwenden. Der urspruengliche Code beruecksichtigte implizit alle Systeme (u.a. GLONASS), deren Ephemeriden jedoch in einem anderen Format vorliegen und in den GPS-spezifischen Kepler-Feldern NaN-Werte erzeugen.

Behoben: `berechne_trajektorie()` filtert konsequent auf SV-Kennungen, die mit `"G"` beginnen.

Fehler 6 - Falsche/synthetische Daten in Aufgabe c), d) und e).

Aufgabe c)/d) rechneten mit der Position aus dem synthetischen Test (Aufgabe a/b) statt mit echten, pro Epoche aus der statischen Aufzeichnung berechneten Positionen. Aufgabe e) uebergab die *statische* statt der *dynamischen* Aufzeichnung an die Trajektorienberechnung.

Behoben: `position_main.run()` berechnet fuer Aufgabe c) und d) die Position fuer *jede* Epoche der statischen Aufzeichnung (`berechne_trajektorie(data_stat, ...)`) und fuer Aufgabe e) fuer die dynamische Aufzeichnung (`berechne_trajektorie(data_dyn, ...)`).

Aufgabe c) erzeugt zusätzlich eine Kartendarstellung (`karte_erstellen(...)`) zur visuellen Verifikation, wie in der Aufgabenstellung gefordert.

Nach diesen Korrekturen berechnet Aufgabenblock 4 fuer **342/342** Epochen der statischen und **857/857** Epochen der dynamischen Aufzeichnung eine Position. Die Standardabweichung der statischen Positionsschaetzung (Ost/Nord/Oben) liegt im Bereich weniger Meter bis knapp 30 m in der Hoehe - plausibel fuer eine unkorrigierte GNSS-Einzelpunktmessung mit einem Android-Smartphone.

5. Aufgabenblock 5 (Genauigkeit) - Neuimplementierung

Aufgabenblock 5 war im urspruenglichen Code nicht vorhanden und wurde vollstaendig neu im Ordner `Genauigkeit/` implementiert. Als Verfahren zur Steigerung der Positionsgenauigkeit (Aufgabe a) wurde die im Aufgabenblatt vorgeschlagene **Mehrfrequenzverarbeitung** gewaehlt, da beide Rohdatensaetze sowohl L1- (C1C) als auch L5-Pseudorangemessungen (C5Q) fuer GPS enthalten - im Gegensatz zu differenziellem GNSS (DGNSS) wird dafuer keine zusaetzliche Referenzstation benoetigt.

5.1 Ionosphaerenfreie Linearkombination

Die ionosphaerische Laufzeitverzoegerung ist naeherungsweise antiproportional zum Quadrat der Signalfrequenz (Abschnitt 5, Gl. 6):

$$\Delta I^{(i)} \approx 40.3 \cdot \text{TEC} / f^2$$

Damit laesst sich der ionosphaerische Fehler durch eine gewichtete Differenz zweier Frequenzmessungen eliminieren:

$$pr_{IF} = (f_1^2 \cdot pr_{L1} - f_2^2 \cdot pr_{L5}) / (f_1^2 - f_2^2)$$

mit $f_1 = 1575.42$ MHz (GPS L1) und $f_2 = 1176.45$ MHz (GPS L5). Implementiert in `Genauigkeit/ionofrei.py`. Ist fuer einen Satelliten keine L5-Messung vorhanden, faellt die Funktion auf die unkorrigierte L1-Pseudorange zurueck, damit der Satellit nicht komplett aus der Loesung herausfaellt.

Die Funktion `pseudorange_ionosphaerenfrei(epoche)` wird als `pseudorange_fn`-Parameter an die bereits aus Block 4 bekannte `berechne_trajektorie()` uebergeben - die eigentliche Positionsschaetzung (Newton-Verfahren) wird unveraendert wiederverwendet.

5.2 Aufgabe b) - Statistischer Vergleich (statische Messung)

`genauigkeit_b.vergleiche_statistik()` vergleicht Mittelwert und Varianz der ECEF-Positionsschaetzungen aus Abschnitt 4 (nur C1C) und Abschnitt 5 (ionosphaerenfrei). In den vorliegenden Aufzeichnungen sinkt die Varianz durch die Zweifrequenzkombination nicht, sondern steigt leicht an. Das ist physikalisch nachvollziehbar: Die zivile L5-Messung eines Android-Smartphones ist rauschbehafteter als L1, und die ionosphaerenfreie Kombination verstaerkt das Messrauschen um einen festen Faktor (fuer GPS L1/L5 ca. Faktor 3), waehrend die ionosphaerische Restfehlerkomponente auf der kurzen Basislinie ueber die Aufzeichnungsdauer ohnehin klein war. Der reale Effekt haengt somit stark von der Signalqualitaet des jeweiligen Empfaengers ab.

5.3 Aufgabe c) - Optischer Vergleich (dynamische Messung)

`genauigkeit_c.vergleiche_dynamisch()` stellt beide dynamischen Trajektorien (Abschnitt 4 in Blau, Abschnitt 5 in Rot) auf derselben interaktiven Karte (`vergleich_dynamische_trajektorie.html`) dar, sodass sie unmittelbar optisch verglichen werden koennen.

6. Ausfuehrung

```
python main.py
```

Das Programm laedt die Rohdaten aus `Rohdateien/` und arbeitet die vier Aufgabenbloেকে nacheinander ab. Da mehrere Diagramme mit `matplotlib` (`plt.show()`) blockierend angezeigt werden, muss jedes Fenster geschlossen werden, damit das Programm fortfaehrt. Als Ergebnis werden u.a. folgende Kartendateien im Projektverzeichnis erzeugt: `statische_trajektorie.html`, `dynamische_trajektorie.html` und `vergleich_dynamische_trajektorie.html`.

Benoetigte Pakete: `georinex`, `numpy`, `pandas`, `matplotlib`, `folium`.

7. Repository-Bereinigung

- `__pycache__`-Verzeichnisse aus der Versionsverwaltung entfernt und `.gitignore` ergaenz.
- `einlesung.py` entfernt (unbenutztes Duplikat frueher Experimente, funktional durch `Orbit/orbit_basis.py` ersetzt).
- Leere/versehentliche Datei `init` im Projektwurzelvezeichnis entfernt.
- `Positionsbestimmung/visualisiere_trajektorie.py` durch `Positionsbestimmung/trajektorie.py` ersetzt (Berechnung und Kartendarstellung sauber getrennt, wiederverwendbar fuer Block 4 und Block 5).